```
// g++ -O2 -std=c++17 b2_verify_interval.cpp -lgmp -o b2_verify
// ./b2_verify b2_rectangle_bounds.csv > b2_verification_output.txt
// Exact rational interval certificate for (2.80)-(2.83).
// All 2600 w-cells and the complete alpha interval are checked for each
// statistic. Every interval operation rounds outwards to multiples of 10^-30.
// Optional first argument: CSV path for all 5200 certified cell bounds.
// The functional estimates supplying (2.61) and (2.80) are from the paper.

#include <gmp.h>
#include <chrono>
#include <iostream>
#include <stdexcept>
#include <string>
#include <utility>
#include <array>
#include <algorithm>
#include <fstream>
#include <vector>

class Q
{
    mpq_t value;

  public:
    Q(long numerator = 0, unsigned long denominator = 1)
    {
        if (denominator == 0)
            throw std::invalid_argument("Zero denominator");
        mpq_init(value);
        mpq_set_si(value, numerator, denominator);
        mpq_canonicalize(value);
    }

    explicit Q(const std::string &text)
    {
        mpq_init(value);
        if (mpq_set_str(value, text.c_str(), 10) != 0)
        {
            mpq_clear(value);
            throw std::invalid_argument("Invalid rational literal");
        }
        mpq_canonicalize(value);
    }

    Q(const Q &other)
    {
        mpq_init(value);
        mpq_set(value, other.value);
    }

    Q(Q &&other) noexcept
    {
        mpq_init(value);
        mpq_swap(value, other.value);
    }

    Q &operator=(Q other) noexcept
    {
        mpq_swap(value, other.value);
        return *this;
    }

    ~Q()
    {
        mpq_clear(value);
    }

    mpq_srcptr raw() const
    {
        return value;
    }
```

```
    mpq_ptr raw()
    {
        return value;
    }

    friend Q operator+(const Q &a, const Q &b)
    {
        Q result;
        mpq_add(result.value, a.value, b.value);
        return result;
    }

    friend Q operator-(const Q &a, const Q &b)
    {
        Q result;
        mpq_sub(result.value, a.value, b.value);
        return result;
    }

    friend Q operator*(const Q &a, const Q &b)
    {
        Q result;
        mpq_mul(result.value, a.value, b.value);
        return result;
    }

    friend Q operator/(const Q &a, const Q &b)
    {
        if (mpq_sgn(b.value) == 0)
            throw std::invalid_argument("Division by zero");
        Q result;
        mpq_div(result.value, a.value, b.value);
        return result;
    }

    friend bool operator<(const Q &a, const Q &b)
    {
        return mpq_cmp(a.value, b.value) < 0;
    }

    friend bool operator==(const Q &a, const Q &b)
    {
        return mpq_cmp(a.value, b.value) == 0;
    }

    friend std::ostream &operator<<(std::ostream &out, const Q &a)
    {
        char *buffer = mpq_get_str(nullptr, 10, a.value);
        std::string text(buffer);
        void (*release)(void *, size_t);
        mp_get_memory_functions(nullptr, nullptr, &release);
        release(buffer, text.size() + 1);
        return out << text;
    }
};

Q power(Q base, unsigned exponent)
{
    Q result(1);
    while (exponent)
    {
        if (exponent & 1U)
            result = result * base;
        base = base * base;
        exponent >>= 1;
    }
    return result;
}

const Q grid(std::string("1000000000000000000000000000000"));
const Q beta(13, 10000);
long long divisor_checks = 0;
```

```
Q rounded(const Q &x, bool upwards)
{
    const Q scaled = x * grid;
    Q result;
    if (upwards)
        mpz_cdiv_q(mpq_numref(result.raw()), mpq_numref(scaled.raw()),
                   mpq_denref(scaled.raw()));
    else
        mpz_fdiv_q(mpq_numref(result.raw()), mpq_numref(scaled.raw()),
                   mpq_denref(scaled.raw()));
    mpz_set(mpq_denref(result.raw()), mpq_numref(grid.raw()));
    mpq_canonicalize(result.raw());
    return result;
}

Q down(const Q &x)
{
    return rounded(x, false);
}

Q up(const Q &x)
{
    return rounded(x, true);
}

Q absolute(const Q &x)
{
    return x < Q(0) ? Q(0) - x : x;
}

struct Interval
{
    Q lo, hi;

    Interval(const Q &x = Q(0)) : lo(x), hi(x)
    {
    }

    Interval(const Q &lower, const Q &upper) : lo(lower), hi(upper)
    {
        if (hi < lo)
            throw std::runtime_error("Reversed interval");
    }
};

Interval add(const Interval &a, const Interval &b)
{
    return {down(a.lo + b.lo), up(a.hi + b.hi)};
}

Interval negative(const Interval &a)
{
    return {down(Q(0) - a.hi), up(Q(0) - a.lo)};
}

Interval subtract(const Interval &a, const Interval &b)
{
    return add(a, negative(b));
}

Interval multiply(const Interval &a, const Interval &b)
{
    const std::array<Q, 4> products = {
        a.lo * b.lo, a.lo * b.hi, a.hi * b.lo, a.hi * b.hi};
    return {down(*std::min_element(products.begin(), products.end())),
            up(*std::max_element(products.begin(), products.end()))};
}

Interval scale(const Q &c, const Interval &a)
{
    return multiply(Interval(c), a);
```

```
}

Interval divide(const Interval &a, const Interval &b)
{
    ++divisor_checks;
    if (!(b.hi < Q(0) || Q(0) < b.lo))
        throw std::runtime_error("Divisor interval contains zero");
    const Interval reciprocal(down(Q(1) / b.hi), up(Q(1) / b.lo));
    return multiply(a, reciprocal);
}

Interval interval_power(const Interval &a, unsigned exponent)
{
    Interval result(Q(1));
    for (unsigned k = 0; k < exponent; ++k)
        result = multiply(result, a);
    return result;
}

Q absolute_upper(const Interval &a)
{
    return std::max(absolute(a.lo), absolute(a.hi));
}

// Exact positive Taylor sum and an exact geometric bound on its tail.
Interval exponential_at(const Q &x)
{
    if (x < Q(0) || Q(13, 5) < x)
        throw std::runtime_error("Exponential argument outside [0,13/5]");
    Q term(1), sum(1);
    for (unsigned k = 1; k <= 24; ++k)
    {
        term = term * x / Q(k);
        sum = sum + term;
    }
    const Q denominator = Q(1) - x / Q(26);
    if (!(Q(0) < denominator))
        throw std::runtime_error("Invalid exponential-tail denominator");
    const Q tail = (term * x / Q(25)) / denominator;
    return {down(sum), up(sum + tail)};
}

// Coefficients in ascending powers of T, generated by
// v=(1-T^2)/4 and d/dw=(1-T^2)/2*d/dT.
std::array<std::vector<Q>, 6> derivative_polynomials()
{
    std::array<std::vector<Q>, 6> result;
    result[0] = {Q(1, 4), Q(0), Q(-1, 4)};
    for (unsigned j = 1; j < result.size(); ++j)
    {
        const auto &previous = result[j - 1];
        std::vector<Q> next(previous.size() + 1, Q(0));
        for (unsigned k = 1; k < previous.size(); ++k)
        {
            const Q coefficient = Q(k) * previous[k] / Q(2);
            next[k - 1] = next[k - 1] + coefficient;
            next[k + 1] = next[k + 1] - coefficient;
        }
        result[j] = std::move(next);
    }
    return result;
}

Interval horner(const std::vector<Q> &coefficients, const Interval &x)
{
    Interval result(Q(0));
    for (auto it = coefficients.rbegin(); it != coefficients.rend(); ++it)
        result = add(multiply(result, x), Interval(*it));
    return result;
}

long factorial(unsigned n)
```

```
{
    long result = 1;
    for (unsigned k = 2; k <= n; ++k)
        result *= k;
    return result;
}

long falling(unsigned n, unsigned k)
{
    long result = 1;
    for (unsigned j = 0; j < k; ++j)
        result *= n - j;
    return result;
}

long binomial(unsigned n, unsigned k)
{
    return factorial(n) / (factorial(k) * factorial(n - k));
}

const std::array<Q, 8> sector_C = {
    Q(0), Q(2), Q(3), Q(8), Q(26), Q(106), Q(533), Q(3134)};
const std::array<Q, 7> sector_D = {
    Q(1, 2), Q(5, 12), Q(1, 2), Q(1), Q(2), Q(6), Q(17)};
const std::array<Q, 5> arc_power = {Q(1), Q(3, 2), Q(2), Q(3), Q(4)};

Q weighted_C(unsigned d, const Q &x)
{
    if (d < 1 || d > 7 || x < Q(0))
        throw std::runtime_error("Invalid weighted-sector index or argument");
    const Q a = Q(119, 100) + Q(27, 50) * x;
    const Q H = Q(2 * factorial(d)) *
                (Q(1) / power(a, d + 1) + Q(5, 12 * d) / power(a, d));
    return std::min(sector_C[d], H);
}

Q weighted_D(unsigned j, const Q &x)
{
    if (j > 6)
        throw std::runtime_error("Invalid derivative-sector index");
    return std::min(sector_D[j], weighted_C(j + 1, x));
}

// Positive scalar bounds here are evaluated as exact rationals. Only
// their final upper bound is quantized; no intermediate approximation occurs.
Q error_upper(unsigned j, const Interval &W,
              const std::array<Q, 6> &Vabs, unsigned statistic)
{
    const unsigned k = j + 1;
    const Q lambda(2 * statistic - 1, 2);
    const Q ell = statistic == 1 ? Q(1) : Q(3, 2);
    const std::array<Q, 3> h = statistic == 1
        ? std::array<Q, 3>{Q(1, 3), Q(2, 3), Q(1, 2)}
        : std::array<Q, 3>{Q(7, 3), Q(8, 3), Q(5, 2)};
    const Q x = statistic == 1 ? std::max(Q(0), W.lo - beta) : W.lo;
    const Q B = h[j] / Q(2) * weighted_D(j + 2, x) +
                lambda * ell * (weighted_D(j + 3, x) + weighted_D(j + 2, x)) +
                power(lambda, 2) / Q(2) *
                    (weighted_C(k + 4, x) + Q(2) * weighted_C(k + 3, x) +
                     weighted_C(k + 2, x));
    std::array<Q, 4> amplitude;
    for (unsigned r = 1; r <= 4; ++r)
    {
        Q sum(0);
        for (unsigned a = 0; a <= std::min(r, k); ++a)
        {
            const Q derivative = r < 4 ? Vabs[j + r - a]
                : arc_power[k - a] * weighted_D(j + r - a, W.lo);
            sum = sum + Q(binomial(r, a) * falling(k, a)) *
                        power(W.hi, r - a) * derivative;
        }
        amplitude[r - 1] = sum;
```

```
    }
    const Q P = amplitude[0] + amplitude[1] + amplitude[2] + amplitude[3] / Q(8);
    const Q G1 = Q(k + 1) * Vabs[j + 2] + W.hi * Vabs[j + 3];
    Q G2(0);
    for (unsigned a = 0; a <= 2; ++a)
        G2 = G2 + Q(binomial(2, a) * falling(k + 1, a)) * arc_power[k + 1 - a] *
                  power(W.hi, 2 - a) * weighted_D(j + 4 - a, W.lo);
    return up(P + lambda * (Q(3) * G1 + G2) + (Q(2) + Q(10) * beta) * B + Q(3));
}

Q certify_rectangle(const Interval &W, const Interval &T,
                    const std::array<Interval, 6> &V, unsigned statistic)
{
    const Interval alpha(Q(41, 25), Q(33, 20));
    const Interval twice_alpha = scale(Q(2), alpha);
    const Q lambda(2 * statistic - 1, 2);
    const Interval eta = add(Interval(lambda),
                             divide(multiply(W, W), scale(Q(4), alpha)));
    const std::array<Interval, 3> A = {
        subtract(subtract(divide(negative(V[0]), alpha),
                          divide(multiply(scale(Q(3), W), V[1]), twice_alpha)),
                 multiply(eta, V[2])),
        subtract(subtract(divide(scale(Q(-5), V[1]), twice_alpha),
                          divide(multiply(scale(Q(2), W), V[2]), alpha)),
                 multiply(eta, V[3])),
        subtract(subtract(divide(scale(Q(-9), V[2]), twice_alpha),
                          divide(multiply(scale(Q(5), W), V[3]), twice_alpha)),
                 multiply(eta, V[4]))};
    // The closed form (2.78) controls the linear correction uniformly.
    const Interval inside = subtract(
        add(divide(Interval(Q(-13)), alpha),
            divide(multiply(multiply(Interval(Q(15)), W), T), alpha)),
        multiply(scale(Q(10), eta), subtract(Interval(Q(1)), scale(Q(6), V[0]))));
    const Interval linear = add(divide(interval_power(V[0], 2), twice_alpha),
                                multiply(interval_power(V[0], 3), inside));
    std::array<Q, 6> Vabs;
    std::array<Q, 3> Aabs, E, U;
    for (unsigned j = 0; j < 6; ++j)
        Vabs[j] = absolute_upper(V[j]);
    for (unsigned j = 0; j < 3; ++j)
    {
        Aabs[j] = absolute_upper(A[j]);
        E[j] = error_upper(j, W, Vabs, statistic);
        U[j] = up(Vabs[j] + beta * Aabs[j]);
    }
    if (!(Q(0) < V[0].lo))
        throw std::runtime_error("Positive lower bound for v was not obtained");
    Interval lower = scale(Q(2), interval_power(Interval(V[0].lo), 3));
    lower = add(lower, Interval(beta * std::min(linear.lo, Q(0))));
    const std::array<Q, 4> losses = {
        power(beta, 2) * Aabs[0] * Aabs[2],
        power(beta, 2) * (Q(2) * U[1] * E[1] + U[0] * E[2] + U[2] * E[0]),
        power(beta, 4) * (power(E[1], 2) + E[0] * E[2]),
        power(beta, 2) / Q(4) * power(U[2] + power(beta, 2) * E[2], 2)};
    for (const auto &loss : losses)
        lower = subtract(lower, Interval(loss));
    return lower.lo;
}
std::string fixed_decimal(const Q &x)
{
    const Q scaled = x * grid;
    if (mpz_cmp_ui(mpq_denref(scaled.raw()), 1) != 0)
        throw std::runtime_error("Value is not on the 10^-30 grid");
    char *raw = mpz_get_str(nullptr, 10, mpq_numref(scaled.raw()));
    std::string digits(raw);
    void (*release)(void *, size_t);
    mp_get_memory_functions(nullptr, nullptr, &release);
    release(raw, digits.size() + 1);
    const bool minus = !digits.empty() && digits[0] == '-';
    if (minus)
        digits.erase(0, 1);
    if (digits.size() <= 30)
```

```
        digits.insert(0, 31 - digits.size(), '0');
    digits.insert(digits.size() - 30, 1, '.');
    return minus ? "-" + digits : digits;
}

void require(bool condition, const char *message)
{
    if (!condition)
        throw std::runtime_error(message);
}

void self_checks()
{
    const Q third(-1, 3), quantum = Q(1) / grid;
    require(!(third < down(third)) && !(up(third) < third) &&
                up(third) - down(third) == quantum,
            "Negative rational outward rounding failed");
    const Interval product = multiply(Interval(Q(-2), Q(3)), Interval(Q(-4), Q(5)));
    require(product.lo == Q(-12) && product.hi == Q(15),
            "Signed interval multiplication failed");
    const Interval quotient = divide(Interval(Q(1), Q(2)), Interval(Q(-4), Q(-2)));
    require(quotient.lo == Q(-1) && quotient.hi == Q(-1, 4),
            "Negative interval division failed");
    const Interval square = interval_power(Interval(Q(2), Q(3)), 2);
    require(square.lo == Q(4) && square.hi == Q(9), "Interval power failed");
    const Interval one = exponential_at(Q(0));
    require(one.lo == Q(1) && one.hi == Q(1), "Exponential enclosure at zero failed");
    bool rejected = false;
    try
    {
        divide(Interval(Q(1)), Interval(Q(-1), Q(1)));
    }
    catch (const std::runtime_error &)
    {
        rejected = true;
    }
    require(rejected, "Zero-containing divisor was accepted");
    const auto polynomials = derivative_polynomials();
    require(polynomials[2][0] == Q(-1, 8) && polynomials[5][0] == Q(0),
            "Logistic derivative recurrence failed");
    divisor_checks = 0;
    std::cout << "SELF_CHECK_PASS tests=7\n";
}

int main(int argc, char **argv)
{
    try
    {
        if (argc > 2)
            throw std::runtime_error("Usage: b2_verify [rectangle_bounds.csv]");
        const auto start = std::chrono::steady_clock::now();
        std::cout << "DOMAIN w=0..13/5 alpha=41/25..33/20 beta_max=13/10000\n";
        std::cout << "ARITHMETIC GMP_exact_rational outward_grid=10^-30 exp_degree=24\n";
        self_checks();
        const auto polynomials = derivative_polynomials();
        std::array<Interval, 2601> exponentials;
        for (int a = 0; a <= 2600; ++a)
            exponentials[a] = exponential_at(Q(a, 1000));
        std::cout << "EXPONENTIAL_ENCLOSURES_READY endpoints=2601\n";
        std::ofstream csv;
        if (argc == 2)
        {
            csv.open(argv[1]);
            if (!csv)
                throw std::runtime_error("Cannot open rectangle CSV");
            csv << "statistic,cell,w_lower,w_upper,alpha_lower,alpha_upper,lower_bound\n";
        }
        std::array<Q, 2> minimum;
        std::array<int, 2> minimum_cell = {0, 0};
        long rectangles = 0;
        for (int a = 0; a < 2600; ++a)
        {
```

```
            const Interval W(Q(a, 1000), Q(a + 1, 1000));
            const Q left = exponentials[a].lo, right = exponentials[a + 1].hi;
            require(Q(0) < left + Q(1) && Q(0) < right + Q(1),
                    "Invalid logistic-transform denominator");
            const Interval T(down((left - Q(1)) / (left + Q(1))),
                             up((right - Q(1)) / (right + Q(1))));
            std::array<Interval, 6> V;
            for (unsigned j = 0; j < 6; ++j)
                V[j] = horner(polynomials[j], T);
            for (unsigned statistic = 1; statistic <= 2; ++statistic)
            {
                const Q lower = certify_rectangle(W, T, V, statistic);
                if (!(Q(3, 10000) < lower))
                {
                    std::cerr << "RECTANGLE_FAIL statistic=" << statistic
                              << " cell=" << a << " lower=" << lower << '\n';
                    return 1;
                }
                const unsigned k = statistic - 1;
                if (a == 0 || lower < minimum[k])
                {
                    minimum[k] = lower;
                    minimum_cell[k] = a;
                }
                ++rectangles;
                if (csv.is_open())
                    csv << (statistic == 1 ? "crank" : "rank") << ',' << a << ','
                        << W.lo << ',' << W.hi << ",41/25,33/20,"
                        << fixed_decimal(lower) << '\n';
            }
            if ((a + 1) % 200 == 0)
                std::cout << "BLOCK_PASS cells=" << a - 199 << ".." << a
                          << " rectangles=400 cumulative=" << rectangles << std::endl;
        }
        if (csv.is_open())
        {
            csv.flush();
            require(bool(csv), "CSV write failed");
        }
        require(rectangles == 5200, "Rectangle coverage count mismatch");
        require(Q(465, 1000000) < minimum[0], "Claimed crank minimum not certified");
        require(Q(407, 1000000) < minimum[1], "Claimed rank minimum not certified");
        std::cout << "MINIMUM crank lower=" << fixed_decimal(minimum[0])
                  << " cell=" << minimum_cell[0] << '\n';
        std::cout << "MINIMUM rank lower=" << fixed_decimal(minimum[1])
                  << " cell=" << minimum_cell[1] << '\n';
        std::cout << "PASS crank_minimum>465/10^6\n";
        std::cout << "PASS rank_minimum>407/10^6\n";
        std::cout << "PASS L_iota(w)>3/10000 on every complete rectangle\n";
        std::cout << "PASS cells=2600_per_statistic rectangles=5200 alpha_interval_complete\n";
        const auto elapsed = std::chrono::duration_cast<std::chrono::microseconds>(
                                 std::chrono::steady_clock::now() - start).count();
        std::cout << "COMPLETE_PASS rectangles=" << rectangles
                  << " divisor_checks=" << divisor_checks
                  << " elapsed_us=" << elapsed << '\n';
        return 0;
    }
    catch (const std::exception &error)
    {
        std::cerr << "CERTIFICATE_FAIL " << error.what() << '\n';
        return 2;
    }
}
```